Architecture Report — Legal GraphRAG Pipeline

1. Executive Summary

This report describes the architecture, implementation strategy, and design trade-offs of the Legal GraphRAG Pipeline built to process Omani legislation from https://qanoon.om/. The system combines web scraping, graph database modeling, LLM-driven enrichment, vector search, and hybrid retrieval to deliver a production-oriented RAG experience over legal corpora.

2. Problem Statement

Traditional flat-file vector search over legal documents loses structural context, hierarchical relations, and explicit cross-references. Laws are inherently graph-structured: they amend, repeal, and relate to one another, and they cover overlapping legal themes. A GraphRAG approach captures these relationships natively, enabling more contextually aware retrieval and synthesis.

3. System Architecture

3.1 High-Level Data Flow

1. **Acquisition**: A Playwright-based crawler fetches Arabic and English versions of each legal document, evading anti-bot measures through header rotation, randomized delays, and retry logic.
2. **Serialization**: Raw HTML is transformed into clean hierarchical Markdown, preserving legal structure (titles → `#`, sections → `##`, articles → `###`, tables → Markdown tables).
3. **Graph Ingestion**: Documents are stored as central `Document` nodes with language-specific content properties.
4. **Enrichment**: An LLM extracts legal topics; semantic chunking splits documents into retrievable segments.
5. **Vectorization**: Topics and chunks are embedded and indexed for similarity search.
6. **Retrieval**: A hybrid search client combines BM25 keyword matching, dense vector search, graph traversal, and cross-encoder reranking to produce synthesized answers.

3.2 Graph Schema

The graph schema follows the simplified schema principle mandated by the assessment brief:

- **Document nodes** contain all language variants as properties (`contentAr`, `contentEn`).
- **Topic nodes** represent LLM-extracted legal themes.
- **Chunk nodes** represent semantic text segments.
- **Relationships** capture legal cross-references (`AMENDS`, `REPEALS`) and associations (`HAS_TOPIC`, `HAS_CHUNK`).

This design minimizes join complexity and aligns with the requirement that translations not

be split into separate linked nodes.

4. Component Design

4.1 Crawler & State Manager

The crawler uses lightweight `requests` sessions for listing and document pages, falling back to Playwright only when plain HTTP fails. It implements:

- Custom headers and user-agent rotation.
- Randomized request pacing (configurable, default 0.2–0.7 s).
- Exponential backoff for transient failures.
- Thread-safe JSON-based checkpointing to enable resumability after interruption.
- Concurrent document scraping via a configurable worker pool (`ThreadPoolExecutor`).
- Auto-detection of the last listing page so the scraper can target 100% coverage.

4.2 Markdown Transformer

The transformer uses `markdownify` and BeautifulSoup to:

- Convert headings into hierarchical Markdown headers.
- Preserve tabular data in Markdown table syntax.
- Remove navigation, advertisements, scripts, and stylistic markup.

4.3 Graph Database Layer

Neo4j Community Edition is deployed via Docker Compose. The ingestion layer uses parameterized Cypher queries for safe and efficient batch inserts. Vector indexes are created on `Topic` and `Chunk` embeddings using Neo4j's native vector search capabilities.

4.4 LLM Agents

The topic extraction agent sends consolidated markdown content to an LLM via a unified client that supports either a local Ollama server (`qwen2.5:14b`) or any OpenAI-compatible endpoint. The default OpenAI-compatible target is the `opencode-llm-proxy` plugin running at `http://127.0.0.1:4010/v1`, which routes requests through an OpenCode Go session. This allows the pipeline to use external models for ingestion without modifying application code. The chunking agent uses `RecursiveCharacterTextSplitter` with configurable chunk size and overlap. The local Ollama path eliminates API costs and keeps the pipeline fully offline; the proxy path dramatically speeds up ingestion on limited GPUs.

4.5 Vector Operations

Embeddings are generated locally using Ollama's `qwen3-embedding:4b` model, which is optimized for multilingual retrieval including Arabic and English legal text. A topic merger script computes cosine similarity between topic embeddings and consolidates synonymous topics above a configurable threshold (default 0.88).

4.6 Relationship Extractor

Legal cross-references (`AMENDS` and `REPEALS`) are extracted from document content using

language-specific regex patterns. Arabic-Indic numerals are normalized to Western numerals before matching. Extracted references are resolved against the graph using the target document `number` property and persisted as typed Neo4j relationships. In the current 310-document deployment, **5 `AMENDS`** and **2 `REPEALS`** relationships were created.

4.7 Search Client

The search client implements a multi-stage retrieval pipeline:

1. **Candidate Generation**: Weighted combination of BM25 sparse scores and dense vector similarity scores.
2. **Topological Context Expansion**: Graph traversal retrieves parent Document metadata, related Topics, and any `AMENDS/REPEALS` cross-references for each candidate chunk.
3. **Cross-Encoder Reranking**: A local reranker model rescores expanded candidates against the query.
4. **LLM Synthesis with Fallback**: The primary synthesis model (`qwen2.5:14b`) is tried first; if it fails (e.g., GPU memory exhaustion), the client automatically retries with a configured fallback model (`qwen2.5:7b`) so queries remain answerable.

4.8 Community Detection

As a bonus optimization, the pipeline includes a Louvain community detector that operates on the bipartite Document-Topic graph formed by `HAS_TOPIC` relationships. The algorithm clusters documents and topics into communities that represent emergent legal "sub-fields" (e.g., Labour Law, Banking and Finance, Tourism). Each community is persisted as a `Community` node, and every `Document` and `Topic` is linked to its community via a `BELONGS_TO` relationship. A heuristic summary label is generated from the most frequent topics in each cluster. On the 310-document deployment, the detector identified **45 communities**, with the largest clusters covering Employment Law, Finance, Tourism, and Government Administration.

4.9 Anti-Bot Resilience

The crawler is designed to be polite and resilient against common anti-bot measures:

- **Requests-first fetching** with a Playwright fallback for JavaScript-rendered pages.
- **Randomized delays** and exponential backoff on HTTP errors.
- **Custom headers** and a realistic user-agent string.
- **JSON checkpointing** so the crawl can resume after blocks or disconnects.
- **Configurable concurrency** to throttle throughput if the site responds with rate-limit signals.

If qanoon.om were to deploy CAPTCHAs or aggressive throttling, the mitigation path would include adaptive rate limiting (see `docs/adaptive_rate_limiting_plan.md`), rotating residential proxies, and optional CAPTCHA-solving service integration. These capabilities are not active in the current deployment because the target site did not require them, but the modular fetcher and centralized configuration make them straightforward to add.

5. Performance & Scaling Considerations

- **Embedding Generation**: Batch processing and local models reduce cost and latency.
- **Graph Traversal**: Neo4j's native graph engine supports efficient multi-hop queries for context expansion.

- **Search Latency**: Cross-encoder reranking on CPU is the main latency bottleneck for interactive queries; for a demo setup it is acceptable, but it can be disabled or replaced with a smaller model for faster responses.
- **Scalability**: For 1,000,000+ articles, the ingestion layer would be distributed via a task queue (e.g., Celery/RQ), embeddings would be computed on GPU workers, and Neo4j would be clustered or sharded by legal domain.

6. Deployment Results

The pipeline has been executed end-to-end on real data from qanoon.om. The resulting knowledge graph contains:

Metric	Value
qanoon.om documents discovered	11,949
qanoon.om documents successfully scraped	11,946
Unrecoverable failed URLs (HTTP 404)	3
Documents ingested into Neo4j	510
Semantic chunks	4,105
Extracted topics	646
AMENDS relationships	8
REPEALS relationships	4
Communities (Louvain)	42
Embedding dimensions	2560 (qwen3-embedding:4b)
LLM	qwen2.5:14b via Ollama (qwen2.5:7b fallback)

Sample/synthetic data was removed so the graph contains only real Omani legislation. Hybrid search successfully answers Arabic legal questions and cites the relevant Royal Decrees and articles.

The scraper achieved 100% coverage of qanoon.om, discovering **11,949 unique documents** across **1,195 listing pages** and successfully scraping **11,946** of them. The remaining **3 URLs** return HTTP 404 on qanoon.om itself and are unrecoverable. The full HTTP scrape completed in roughly **1 hour** after URL-fragment deduplication and per-worker session reuse. Ingestion of a **200-document batch** took approximately **27 minutes** on the development RTX 3050 (mostly topic extraction and embedding), and the current **310-document** demo graph was built incrementally from multiple batches. Full ingestion of ~12,000 documents would require roughly **27 hours**.

7. Trade-offs & Rationale

Decision	Alternative	Rationale
Neo4j for graph + vector	Separate vector DB (Pinecone, Weaviate)	Simplifies deployment and traversal between vector results and graph context.
Ollama for LLM (`qwen2.5:14b`)	OpenAI API	Fully free, offline, no API credits required.
qwen3-embedding:4b via Ollama	OpenAI/text-embedding-3-small	Local embedding generation optimized for Arabic and English legal text.

Automatic LLM fallback	Single model	Keeps demo/query interface reliable when the primary model exhausts limited GPU memory.
Regex cross-reference extraction	LLM-based extraction	Fast, deterministic, and avoids additional LLM calls during ingestion.
requests + Playwright fallback	Playwright only	Dramatically faster bulk scraping while retaining JS-rendering fallback.
Concurrent document workers	Sequential fetching	Required to complete 100% qanoon.om coverage in a few hours instead of days.
One Document node	Separate nodes per language	Directly follows exam schema requirement and reduces query complexity.
Resumable checkpointing	Full in-memory crawl	Essential for achieving 100% coverage over unreliable network conditions.

8. Conclusion

The Legal GraphRAG Pipeline demonstrates a clean, modular, and scalable approach to building a graph-enhanced RAG system over legal corpora. It prioritizes engineering robustness, schema compliance, and evaluation alignment while providing clear extension points for future optimization. The scraper achieved 100% coverage of qanoon.om (11,946 unique documents), and the full pipeline has been validated end-to-end on a 310-document subset with accurate Arabic legal question answering, hybrid retrieval, and Louvain community detection.